

Codec V2 Serialization Feature Specification

Overview

This document defines the serialization features for codec.v2, including configurable strategies for type information, identity, references, and supertype serialization.

1. Serialization Targets

The codec serializes the following metadata alongside EObject features:

Target	Purpose	Applies To
Type	Identifies the EClass of an object	All EObjects
ID	Unique identifier for an object	EObjects with identity
Reference	Points to non-contained objects	EReference (non-containment)
Cross-Doc Containment	Points to contained objects in other documents	EReference (containment, cross-document)
SuperType	Lists supertypes for querying	EObjects (optional)

1.1 EAnnotation and Configuration Parity

Every codec feature that can be configured declaratively via **EAnnotations** on the EMF model should also be configurable programmatically via the **Config Builder** at runtime, and vice versa.

Configuration Levels (Resolution Order, dynamic before static):

Priority	Level	Scope	Description
1 (highest)	Load/Save options	Per-operation	Options passed to <code>resource.save(options)</code> / <code>resource.load(options)</code>
2	ResourceFactory defaults	Per-factory	<code>defaultSaveOptions</code> / <code>defaultLoadOptions</code> on EMF ResourceFactory
3	Codec module config	Per-codec	Jackson module / codec instance configuration
4	Configuration properties	External	System properties, config files
5	EAnnotations	Per-model	Declared in .ecore model (static)
6 (lowest)	Built-in defaults	Global	Hardcoded codec defaults

Key Principle: Every configuration setting (EAnnotation detail keys, Jackson module features) must have a corresponding runtime option key. This allows any setting to be overridden at load/save time without modifying the model or codec configuration.

Core Annotations (source: <http://eclipse.org/fennec/codec>):

Detail Key	Applies To	Purpose
<code>codec.type</code>	EClass, EReference	Marker for type serialization config
<code>codec.id</code>	EClass	Marker for ID serialization config
<code>codec.reference</code>	EReference	Marker for reference serialization config
<code>codec.supertype</code>	EClass	Marker for supertype serialization config
<code>serialize</code>	EStructuralFeature	Skip feature during serialization (false = transient)
<code>key</code>	EStructuralFeature	Custom JSON property name
<code>valueWriterName</code>	EStructuralFeature	Custom value writer reference
<code>valueReaderName</code>	EStructuralFeature	Custom value reader reference

Design Principle: When documenting a configuration option, always show both:

- The **EAnnotation** approach (for model designers)
- The **Config Builder** approach (for runtime customization)

This ensures flexibility: models can define sensible defaults that integrators can override at runtime without modifying the model.

2. Serialization Strategies

2.1 Plain vs Structured Output Format

This is a fundamental concept that applies to all serialization targets (type, ID, reference, supertype).

Definition:

- **Plain:** The property value is a simple type (string, number, boolean) or a simple array
- **Structured:** The property value is a nested object/map

The test: Is the value of a property (e.g., `_type`) a nested JSON object?

Output Format	Value Type	Example
Plain	string	<code>"_type": "Person"</code>
Plain	number	<code>"_type": 3</code>
Plain	array	<code>"_supertypes": ["Entity", "Auditable"]</code>

Output Format	Value Type	Example
Structured	object	<code>"_type": { "schema": "...", "name": "..." }</code>

Why this matters for the builder:

When the serializer builds the property map:

- **Plain strategy:** Write simple value(s) directly to the output map
- **Structured strategy:** Create a nested map as the property value

Example - Type with supertypes enabled:

Plain (SCHEMA_AND_TYPE) - each property has a simple value:

```
{
  "_schema": "http://example.org/person/1.0",
  "_type": "Person",
  "_supertypes": ["Entity", "Auditable"]
}
```

Structured - `_type` value is a nested object containing all info:

```
{
  "_type": {
    "schema": "http://example.org/person/1.0",
    "name": "Person",
    "supertypes": ["Entity", "Auditable"]
  }
}
```

2.2 Strategy Classification

Strategies are classified by their output format:

Strategy	Output Format	Description
<code>NAME</code>	Plain	EClass simple name as string
<code>CLASS</code>	Plain	Instance class name as string
<code>URI</code>	Plain	Full EClass URI as string
<code>MAPPED</code>	Plain	Discriminator value as string
<code>SCHEMA_AND_TYPE</code>	Plain	Separate top-level fields, each with simple values
<code>STRUCTURED</code>	Structured	Nested object with schema, name, etc.
<code>NUMERIC</code>	Structured	Nested object with numeric IDs

2.3 Strategy Availability per Target

Target	Plain Strategies	Structured Strategies
Type	NAME, CLASS, URI, MAPPED, SCHEMA_AND_TYPE	STRUCTURED, NUMERIC
ID	PLAIN	STRUCTURED, NUMERIC
Reference	PLAIN	STRUCTURED, NUMERIC
SuperType	PLAIN	STRUCTURED
Features	(names)	NUMERIC (uses feature IDs as keys)

Note: For ID, Reference, and SuperType, "PLAIN" means the default simple value format (e.g., just the ID string or URI). For Type, the plain format has variants (NAME, CLASS, URI, etc.) that determine *what* string to use.

3. Type Serialization

3.1 Type Strategies

Plain Strategies (string value for `_type`)

Strategy	Value	Example
NAME	EClass name	<code>"_type": "Person"</code>
CLASS	Instance class name	<code>"_type": "org.example.Person"</code>
URI	Full EClass URI	<code>"_type": "http://example.org/person/1.0#//Person"</code>
MAPPED	Discriminator value	<code>"_type": "customer"</code>
SCHEMA_AND_TYPE	Name + separate schema field	<code>"_schema": "...", "_type": "Person"</code>

MAPPED Strategy (Discriminator-Based Polymorphism)

The MAPPED strategy enables type discrimination based on a value found within the data itself, rather than an explicit type field. This is particularly useful for:

- **IoT/LoRaWAN devices** - where device type is embedded in payload metadata
- **JSON Schema oneOf patterns** - where type is determined by feature presence or values
- **Legacy APIs** - where type information is encoded in application-specific fields

How it works:

1. **Base class** defines the discriminator feature path (which field contains the type indicator)
2. **Concrete classes** define their discriminator value (what value identifies them)

3. **ModelInfoService** builds a reverse lookup map from values → EClasses

Configuration (via annotations or CodecConfig):

```
// On base class (e.g., LoRaWANUplink)
TypeResolutionConfig.builder()
    .strategy(TypeStrategy.MAPPED)
    .typeFeaturePath("deviceInfo.deviceProfileName") // Path to discriminator
    .build();

// On concrete class (e.g., DraginoLSE01Uplink)
TypeResolutionConfig.builder()
    .strategy(TypeStrategy.MAPPED)
    .discriminatorValue("Dragino_LSE01") // Value that identifies this class
    .build();
```

Example Input JSON (no explicit type field):

```
{
  "deviceInfo": {
    "deviceProfileName": "Dragino_LSE01",
    "devEui": "A84041..."
  },
  "temperature": 23.5,
  "humidity": 65
}
```

The deserializer reads `deviceInfo.deviceProfileName`, finds `"Dragino_LSE01"`, and uses the reverse lookup to instantiate `DraginoLSE01Uplink`.

Feature-Based Type Discrimination (JSON Schema oneOf):

For cases where type is determined by feature presence rather than a single discriminator value:

```
TypeResolutionConfig.builder()
    .strategy(TypeStrategy.MAPPED)
    .typeFeaturePath("") // Special marker for feature-based detection
    .typeMap(Map.of(
        "temperature,humidity", "EnvironmentSensor",
        "latitude,longitude", "GPSTracker"
    ))
    .build();
```

The deserializer checks which features are present and matches against the type map.

SCHEMA_AND_TYPE Strategy (Plain)

Split into separate top-level fields (each value is a simple string):

```
{
  "_schema": "http://example.org/person/1.0",
  "_type": "Person"
}
```

Configurable keys:

- `schemaKey`: schema field (default: `_schema`)
- `typeKey`: type field (default: `_type`)

Structured Strategies (object value for `_type`)

STRUCTURED Strategy

Nested object containing type details:

```
{
  "_type": {
    "schema": "http://example.org/person/1.0",
    "name": "Person",
    "supertypes": ["Entity", "Auditable"]
  }
}
```

Configurable keys:

- `typeKey`: outer key (default: `_type`)
- `schemaKey`: schema field inside object (default: `schema`)
- `nameKey`: name field inside object (default: `name`)
- `supertypesKey`: supertypes field inside object (default: `supertypes`, optional)

NUMERIC Strategy

Nested object with schema URI and numeric classifier ID:

```
{
  "_type": { "s": "http://example.org/person/1.0", "c": 3 }
}
```

Configurable keys:

- `typeKey`: outer key (default: `_type`)
- `schemaKey`: schema field (default: `s`)
- `classifierKey`: classifier ID field (default: `c`)

Warning: Classifier IDs are positional and can change when the model evolves. See Section 12.3 for details.

3.2 Mixed Supertype Format

In STRUCTURED format, supertypes use a **smart format**:

- Supertypes from **same schema**: plain name only
- Supertypes from **different schema**: full URI

```
{
  "_type": {
    "schema": "http://example.org/person/1.0",
    "name": "Person",
    "supertypes": ["Entity", "http://audit.org/1.0#//Auditable"]
  }
}
```

This is compact yet unambiguous - `Entity` is from `http://example.org/person/1.0`, while `Auditable` includes its full URI.

3.3 Type Configuration

The configuration defines **keys and format**, not actual values. Values come from the EObject at runtime.

EAnnotation (on EClass)

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.type"/>
    <details key="strategy" value="STRUCTURED"/>
    <details key="typeKey" value="_type"/>
    <details key="include" value="true"/>
  </eAnnotations>
</eClassifiers>
```

Annotation Details:

Key	Values	Default	Description
<code>strategy</code>	NAME, CLASS, URI, MAPPED, STRUCTURED, SCHEMA_AND_TYPE, NUMERIC	URI	Type serialization strategy
<code>typeKey</code>	any string	<code>_type</code>	JSON property name for type
<code>include</code>	true, false	true	Whether to include type info
<code>schemaKey</code>	any string	<code>schema</code>	Key for schema in STRUCTURED
<code>nameKey</code>	any string	<code>name</code>	Key for name in STRUCTURED

Java Builder (Runtime Override)

Minimal (default: URI strategy):

```
TypeSerializationConfig config = TypeSerializationConfig.builder().build();
```

Resulting JSON:

```
{
  "_type": "http://example.org/person/1.0#//Person"
}
```

STRUCTURED format:

```
TypeSerializationConfig config = TypeSerializationConfig.builder()
    .structured()
    .build();
```

Resulting JSON:

```
{
  "_type": {
    "schema": "http://example.org/person/1.0",
    "name": "Person"
  }
}
```

STRUCTURED with supertypes:

```
TypeSerializationConfig config = TypeSerializationConfig.builder()
    .structured()
    .includeSupertypes(true)
    .build();
```

Resulting JSON:

```
{
  "_type": {
    "schema": "http://example.org/person/1.0",
    "name": "Person",
    "supertypes": ["Entity", "Auditable"]
  }
}
```

With custom keys:


```
TypeSerializationConfig config = TypeSerializationConfig.builder()
    .structured()
    .typeKey("$type")
    .schemaKey("ns")
    .nameKey("class")
    .build();
```

Resulting JSON:

```
{
  "$type": {
    "ns": "http://example.org/person/1.0",
    "class": "Person"
  }
}
```

At serialization time:

1. Serializer reads `TypeSerializationConfig` from options (Java or JSON)
2. Extracts actual values from EObject's EClass (schema URI, name, supertypes)
3. Combines config (keys/format) + EObject data (values) → JSON output

4. ID Serialization

4.1 ID Feature Resolution

The serializer determines which features to use as ID:

1. **If `idFeatures` is specified** (via annotation or config) → use those features in the specified order
2. **Otherwise** → use all features with `eID="true"` in the EClass, in declaration order

When `idFeatures` points to an **EReference** (containment), the serializer follows the reference and uses the contained object's ID definition.

Feature Ordering: When multiple features form a combined ID, the order is preserved as specified:

- For `idFeatures="lastName, firstName"` → combined ID is `"Doe-John"` (not `"John-Doe"`)
- For `idFeatures="firstName, lastName"` → combined ID is `"John-Doe"`

This ordering applies to both PLAIN format (concatenation order) and STRUCTURED format (property order in JSON object).

4.2 ID Key Mode

Controls how ID is represented in the JSON output:

Mode	Description	Example (single)	Example (multiple)
ID_ONLY (default)	Only <code>_id</code> key	<code>"_id": "john"</code>	<code>"_id": "John-Doe"</code>
BOTH	<code>_id</code> + original feature names	<code>"_id": "john", "myId": "john"</code>	<code>"_id": "John-Doe", "firstName": "John", "lastName": "Doe"</code>
FEATURE_ONLY	Only original feature names	<code>"myId": "john"</code>	<code>"firstName": "John", "lastName": "Doe"</code>

4.3 ID Format (Plain vs Structured)

Per Section 2.1, ID values can be plain or structured.

Plain Format (default) - Single Feature

EAnnotation:

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="myId" eType=".../EString"
  id="true"/>
</eClassifiers>
```

Config Builder:

```
// No config needed - uses defaults (PLAIN format, ID_ONLY mode)
IdSerializationConfig config = IdSerializationConfig.builder().build();
```

Resulting JSON:

```
{
  "_id": "john"
}
```

Plain Format - Multiple Features

EAnnotation:

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.id"/>
    <details key="idFeatures" value="firstName, lastName, sequence"/>
    <details key="separator" value="-"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="firstName" eType=".../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="lastName" eType=".../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="sequence" eType=".../EString"/>
</eClassifiers>
```

Config Builder:

```
IdSerializationConfig config = IdSerializationConfig.builder()
    .idFeatures("firstName", "lastName", "sequence")
    .separator("-")
    .build();
```

Resulting JSON:

```
{
  "_id": "John-Doe-1"
}
```

Structured Format - Single Feature

EAnnotation:

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.id"/>
    <details key="format" value="STRUCTURED"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="myId" eType="../../../EString"
id="true"/>
</eClassifiers>
```

Config Builder:

```
IdSerializationConfig config = IdSerializationConfig.builder()
    .format(IdFormat.STRUCTURED)
    .build();
```

Resulting JSON:

```
{
  "_id": {
    "myId": "john"
  }
}
```

Structured Format - Multiple Features

EAnnotation:

```

<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.id"/>
    <details key="idFeatures" value="firstName, lastName, sequence"/>
    <details key="separator" value="-"/>
    <details key="format" value="STRUCTURED"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="firstName" eType=".../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="lastName" eType=".../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="sequence" eType=".../EString"/>
</eClassifiers>

```

Config Builder:

```

IdSerializationConfig config = IdSerializationConfig.builder()
    .idFeatures("firstName", "lastName", "sequence")
    .separator("-")
    .format(IdFormat.STRUCTURED)
    .build();

```

Resulting JSON:

```

{
  "_id": {
    "separator": "-",
    "firstName": "John",
    "lastName": "Doe",
    "sequence": "1"
  }
}

```

4.4 ID with EReference (Contained ID Object)

When `idFeatures` points to a containment EReference, the ID is resolved from the contained object.

Example: ExampleIDClass01 (contained IdClass01 has no eID, uses idFeatures)

The contained class IdClass01:

```

<eClassifiers xsi:type="ecore:EClass" name="IdClass01">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="separator" value="_"/>
    <details key="idFeatures" value="userGroup, userId"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="userId" eType="ecore:EDatatype
.../ELong"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="userGroup" eType="ecore:EDatatype
.../EString"/>
</eClassifiers>

```

The parent class pointing to it:

```

<eClassifiers xsi:type="ecore:EClass" name="ExampleIDClass01">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="idFeatures" value="myId"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EReference" name="myId" eType="#//IdClass01"
containment="true"/>
</eClassifiers>

```

Or via Java Builder:

```

IdSerializationConfig config = IdSerializationConfig.builder()
    .idFeatures("myId") // Points to EReference
    .build();

```

Resulting Output:

```

{
  "_type": "...#//ExampleIDClass01",
  "_id": "sales_42",
  "test": "...",
  "myId": {
    "userId": 42,
    "userGroup": "sales"
  }
}

```

Example: ExampleIDClass02 (contained IdClass02 has eID on userId)

```

<eClassifiers xsi:type="ecore:EClass" name="IdClass02">
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="userId" eType="ecore:EDatatype
.../ELong" id="true"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="userGroup" eType="ecore:EDatatype
.../EString"/>
</eClassifiers>

```

Resulting Output:

```

{
  "_type": "...#//ExampleIDClass02",
  "_id": "42",
  "test": "...",
  "myId": {
    "userId": 42,
    "userGroup": "sales"
  }
}

```

4.5 ID Configuration

Annotation Details:

Key	Values	Default	Description
idFeatures	comma-separated feature names	All eID="true" features	Features to use as ID
separator	any string	/	Separator when combining multiple features
keyMode	ID_ONLY, BOTH, FEATURE_ONLY	ID_ONLY	How to represent ID in JSON
format	PLAIN, STRUCTURED	PLAIN	Plain string vs nested object
key	any string	_id	JSON property name for ID
idValueWriterName	writer name	-	Custom value writer for ID
idValueReaderName	reader name	-	Custom value reader for ID

Example 1: Minimal Configuration (Defaults)

EAnnotation:

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.id"/>
    <details key="idFeatures" value="firstName, lastName"/>
    <details key="separator" value="-"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="firstName" eType="../../../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="lastName" eType="../../../EString"/>
</eClassifiers>
```

Config Builder:

```
IdSerializationConfig config = IdSerializationConfig.builder()
    .idFeatures("firstName", "lastName")
    .separator("-")
    .build();

saveOptions.put(CodecOptions.ID_CONFIG, config);
```

Resulting JSON: (keyMode=ID_ONLY, format=PLAIN, key="_id" are defaults)

```
{
  "_id": "John-Doe",
  "firstName": "John",
  "lastName": "Doe"
}
```

Example 2: Non-Default Options (BOTH, STRUCTURED, Custom Key)

EAnnotation:

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.id"/>
    <details key="idFeatures" value="firstName, lastName"/>
    <details key="separator" value="- "/>
    <details key="keyMode" value="BOTH"/>
    <details key="format" value="STRUCTURED"/>
    <details key="key" value="id"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="firstName" eType=".../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="lastName" eType=".../EString"/>
</eClassifiers>
```

Config Builder:

```
IdSerializationConfig config = IdSerializationConfig.builder()
    .idFeatures("firstName", "lastName")
    .separator("- ")
    .keyMode(IdKeyMode.BOTH)
    .format(IdFormat.STRUCTURED)
    .idKey("id")
    .build();

saveOptions.put(CodecOptions.ID_CONFIG, config);
```

Resulting JSON:

```
{
  "id": {
    "separator": "- ",
    "firstName": "John",
    "lastName": "Doe"
  },
  "firstName": "John",
  "lastName": "Doe"
}
```

Example 3: FEATURE_ONLY Mode

EAnnotation:

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.id"/>
    <details key="idFeatures" value="firstName, lastName"/>
    <details key="keyMode" value="FEATURE_ONLY"/>
  </eAnnotations>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="firstName" eType=".../EString"/>
  <eStructuralFeatures xsi:type="ecore:EAttribute" name="lastName" eType=".../EString"/>
</eClassifiers>
```

Config Builder:

```
IdSerializationConfig config = IdSerializationConfig.builder()
    .idFeatures("firstName", "lastName")
    .keyMode(IdKeyMode.FEATURE_ONLY)
    .build();

saveOptions.put(CodecOptions.ID_CONFIG, config);
```

Resulting JSON: (no `_id` property, features are used directly for identification)

```
{
  "firstName": "John",
  "lastName": "Doe"
}
```

4.6 Complete Examples

Example 1: Single ID field (ExampleSingleID)

EClass with `myId` marked `eID="true"`:

Config	Output
Default (ID_ONLY, PLAIN)	<code>{"_id": "john", "test": "..."}"</code>
BOTH, PLAIN	<code>{"_id": "john", "myId": "john", "test": "..."}"</code>
FEATURE_ONLY, PLAIN	<code>{"myId": "john", "test": "..."}"</code>
ID_ONLY, STRUCTURED	<code>{"_id": {"myId": "john"}, "test": "..."}"</code>

Example 2: Multiple ID fields (ExampleMultipleIDs)

EClass with `myId`, `mySecondId`, `myThirdId` all marked `eID="true"`, `separator="-"`:

Config	Output
ID_ONLY, PLAIN	<code>{"_id": "John-Doe-1", "test": "..."}"</code>
BOTH, PLAIN	<code>{"_id": "John-Doe-1", "myId": "John", "mySecondId": "Doe", "myThirdId": "1", "test": "..."}"</code>
ID_ONLY, STRUCTURED	<code>{"_id": {"separator": "-", "myId": "John", "mySecondId": "Doe", "myThirdId": "1"}, "test": "..."}"</code>

Example 3: idFeatures annotation (ExampleMultiple)

EClass with `idFeatures="firstName, lastName", separator="."`:

Config	Output
Default	<code>{"_id": "John.Doe", "firstName": "John", "lastName": "Doe", "test": "..."}"</code>
STRUCTURED	<code>{"_id": {"separator": ".", "firstName": "John", "lastName": "Doe"}, "firstName": "John", ...}"</code>

At serialization time:

1. Resolve ID features (from `idFeatures` config/annotation or `eID="true"` attributes)
2. If feature is EReference → follow and use contained object's ID definition
3. Extract values from EObject
4. Apply format (plain: combine with separator, structured: create object)
5. Apply keyMode (ID_ONLY, BOTH, or FEATURE_ONLY)

5. Reference Serialization (Non-Containment)

5.1 Reference Strategies

PLAIN Strategy

Single value (URI or ID):

```
{
  "employer": "datainmotion"
}
```

Or as URI:

```
{
  "employer": "http://example.org/data#//@companies.0"
}
```

STRUCTURED Strategy

Nested object with type and reference:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0//Company",
    "_ref": "datainmotion"
  }
}
```

Configurable keys:

- `typeKey`: type field (default: `_type`)
- `refKey`: reference field (default: `_ref`)

5.2 Multi-valued References

PLAIN Strategy

```
{
  "employees": ["john-doe", "jane-smith", "bob-wilson"]
}
```

STRUCTURED Strategy

```
{
  "employees": [
    { "_type": "http://example.org/person/1.0//Employee", "_ref": "john-doe" },
    { "_type": "http://example.org/person/1.0//Employee", "_ref": "jane-smith" },
    { "_type": "http://example.org/person/1.0//Manager", "_ref": "bob-wilson" }
  ]
}
```

5.3 Reference Configuration

The configuration defines **keys and format**, not actual values.

EAnnotation (on EReference)

```
<!-- Switch to STRUCTURED format (PLAIN is default) -->
<eStructuralFeatures xsi:type="ecore:EReference" name="employer" eType="#//Company">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.reference"/>
    <details key="format" value="STRUCTURED"/>
  </eAnnotations>
</eStructuralFeatures>
```

Annotation Details:

Key	Values	Default	Description
<code>format</code>	PLAIN, STRUCTURED, NUMERIC	PLAIN	Reference serialization format
<code>typeKey</code>	any string	<code>_type</code>	Key for type in STRUCTURED
<code>refKey</code>	any string	<code>_ref</code>	Key for reference value
<code>includeType</code>	true, false	true	Include type info in STRUCTURED
<code>expand</code>	true, false	false	Serialize full object instead of proxy

Java Builder (Runtime Override)

Minimal (default: PLAIN):

```
ReferenceSerializationConfig config = ReferenceSerializationConfig.builder().build();
```

Resulting JSON:

```
{
  "employer": "acme-corp"
}
```

STRUCTURED format:

```
ReferenceSerializationConfig config = ReferenceSerializationConfig.builder()
    .structured()
    .build();
```

Resulting JSON:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_ref": "acme-corp"
  }
}
```

STRUCTURED with custom keys:

```
ReferenceSerializationConfig config = ReferenceSerializationConfig.builder()
    .structured()
    .typeKey("$type")
    .refKey("$ref")
    .build();
```

Resulting JSON:

```
{
  "employer": {
    "$type": "http://example.org/company/1.0#//Company",
    "$ref": "acme-corp"
  }
}
```

Expand (full object instead of proxy):

```
ReferenceSerializationConfig config = ReferenceSerializationConfig.builder()
    .expand(true)
    .build();
```

Resulting JSON:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_id": "acme-corp",
    "name": "Acme Corporation",
    "industry": "Manufacturing"
  }
}
```

At serialization time:

1. Serializer reads `ReferenceSerializationConfig` from options (Java or JSON)
2. Extracts reference target's type and ID/URI from EObject
3. Combines config (keys/format) + reference data → JSON output

5.4 Proxy and Expand Handling

Default Behavior: Proxy Serialization

Non-containment references are serialized as **proxies by default**. This requires:

- `_type`: Type information (proxy URI doesn't always indicate type)
- `_ref`: Proxy URI for resolution

Only objects with a URI can be serialized as references (standard EMF behavior).

Default proxy output:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_ref": "companies.json#//@companies.0"
  }
}
```

Expand: Inline Serialization

When `expand=true` and the reference is **resolved** (not a proxy), the referenced object is serialized inline instead of as a proxy reference.

Expanded output:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_id": "acme-corp",
    "name": "Acme Corporation",
    "industry": "Manufacturing"
  }
}
```

Expand Configuration:

Level	Option	Default	Description
Codec	<code>expand</code>	<code>false</code>	Enable inline serialization for resolved refs
Codec	<code>expandDepth</code>	<code>1</code>	Max depth for nested expansion
Codec	<code>expandIgnoreBidirectional</code>	<code>true</code>	Skip opposite/bi-directional references
Feature	<code>expand</code>	(codec default)	Override expand for this feature

EAnnotation (on EReference):

```
<eStructuralFeatures xsi:type="ecore:EReference" name="employer" eType="#//Company">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.reference"/>
    <details key="expand" value="true"/>
  </eAnnotations>
</eStructuralFeatures>
```

Java Builder:

Codec level (all non-containments):

```
CodecConfig.builder()
    .expand(true)
    .expandDepth(2)
    .expandIgnoreBidirectional(true)
    .build();
```

Resulting JSON: (employer expanded, but employer.employees skipped if bi-directional)

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_id": "acme-corp",
    "name": "Acme Corporation"
  }
}
```

Feature level override:

```
// Expand all refs, but keep "manager" as proxy
CodecConfig.builder()
  .expand(true)
  .build();

FeatureSerializationConfig.builder()
  .feature("manager")
  .expand(false)
  .build();
```

Resulting JSON:

```
{
  "employer": {
    "_type": "...",
    "_id": "acme-corp",
    "name": "Acme Corporation"
  },
  "manager": {
    "_type": "...",
    "_ref": "employees.json#//@employees.5"
  }
}
```

Type Configuration per Context

Type serialization can be configured differently for each context:

Context	Config	Use Case
Root objects	<code>type(...)</code>	Main serialization
Containments	<code>containmentType(...)</code>	Inline contained objects
References (proxy + expanded)	<code>referenceType(...)</code>	Non-containment refs
Per-feature	<code>feature(...).type(...)</code>	Specific reference

Java Builder:

```

CodecConfig.builder()
    .type(TypeSerializationConfig.builder().build()) // default: URI
    .containmentType(TypeSerializationConfig.builder()
        .strategy(TypeStrategy.NAME) // contained: just name (context clear)
        .build())
    .referenceType(TypeSerializationConfig.builder()
        .strategy(TypeStrategy.URI) // refs: full URI for resolution
        .build())
    .build();

```

Resulting JSON:

```

{
  "_type": "http://example.org/person/1.0#//Person",
  "_id": "john-doe",
  "name": "John Doe",
  "address": {
    "_type": "Address",
    "street": "123 Main St"
  },
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_ref": "companies.json#//@companies.0"
  }
}

```

Bi-directional Reference Handling

When expanding, bi-directional (opposite) references are **ignored by default** to prevent cycles:

```
Person.employer ↔ Company.employees (opposite)
```

If expanding `employer`, the `employees` back-reference in `Company` is skipped.

Configuration:

- `expandIgnoreBidirectional=true` (default): Skip opposite references
- `expandIgnoreBidirectional=false`: Include (use with caution, may cause cycles)

6. Cross-Document Containment

When a contained object is stored in a different document, it is serialized similarly to a reference.

6.1 Cross-Document Containment Strategies

PLAIN Strategy

```
{
  "address": "addresses.json#//@addresses.0"
}
```

STRUCTURED Strategy

```
{
  "address": {
    "_type": "http://example.org/address/1.0#//Address",
    "_ref": "addresses.json#//@addresses.0"
  }
}
```

Or with STRUCTURED type:

```
{
  "address": {
    "_type": {
      "schema": "http://example.org/address/1.0",
      "name": "Address"
    },
    "_ref": "addresses.json#//@addresses.0"
  }
}
```

6.2 Cross-Document Containment Configuration

Cross-document containments use the same configuration as non-containment references (see Section 5.3 and 5.4).

Detection: The serializer detects cross-document containment when:

- The EReference is containment (`isContainment() == true`)
- The contained object's resource differs from the container's resource

Type configuration: Use `containmentType(...)` for cross-document containments:

```
CodecConfig.builder()
    .containmentType(TypeSerializationConfig.builder()
        .strategy(TypeStrategy.URI) // full URI for cross-doc resolution
        .build())
    .build();
```

Resulting JSON:


```
{
  "address": {
    "_type": "http://example.org/address/1.0#//Address",
    "_ref": "addresses.json#//@addresses.0"
  }
}
```

7. SuperType Serialization

SuperType serialization has two orthogonal dimensions:

- **Selection** (ALL, SINGLE, NONE) - which supertypes to include
- **Format** (PLAIN, STRUCTURED) - per Section 2.1, how to represent each entry

7.1 SuperType Format

PLAIN Format

Array of plain type identifiers (strings):

```
{
  "_supertypes": ["Entity", "Auditable", "Timestamped"]
}
```

Or as URIs:

```
{
  "_supertypes": [
    "http://example.org/base/1.0#//Entity",
    "http://example.org/audit/1.0#//Auditable"
  ]
}
```

STRUCTURED Format

Array of structured type objects:

```
{
  "_supertypes": [
    { "schema": "http://example.org/base/1.0", "name": "Entity" },
    { "schema": "http://example.org/audit/1.0", "name": "Auditable" }
  ]
}
```

7.2 Mixed Format (Smart Compression)

Similar to type info, supertypes use a **smart format**:

- Supertypes from **same schema** as the object: plain name only
- Supertypes from **different schema**: full URI

```
{
  "_supertypes": ["Entity", "NamedElement", "http://audit.org/1.0//Auditable"]
}
```

7.3 SuperType Configuration

The configuration defines **keys and format**, not actual values.

EAnnotation (on EClass)

```
<eClassifiers xsi:type="ecore:EClass" name="Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.supertype"/>
    <details key="enabled" value="true"/>
    <details key="selection" value="ALL"/>
    <details key="supertypesKey" value="_supertypes"/>
  </eAnnotations>
</eClassifiers>
```

Annotation Details:

Key	Values	Default	Description
enabled	true, false	false	Enable supertype serialization
selection	ALL, ALL_EMF, SINGLE, NONE	ALL (when enabled)	Which supertypes to include
supertypesKey	any string	_supertypes	JSON property name
format	PLAIN, STRUCTURED	PLAIN	Per Section 2.1, plain strings vs objects
schemaKey	any string	schema	Key for schema in STRUCTURED
nameKey	any string	name	Key for name in STRUCTURED

Selection Values:

Value	Description
ALL	All domain model supertypes (excludes EMF base classes like EObject)
ALL_EMF	All supertypes including EMF base classes (EObject, EModelElement, etc.)

Value	Description
<code>SINGLE</code>	Only the immediate/direct supertype
<code>NONE</code>	No supertypes (equivalent to <code>enabled=false</code>)

Java Builder (Runtime Override)

Minimal (all defaults: **PLAIN** format, **ALL** selection):

```
SuperTypeSerializationConfig config = SuperTypeSerializationConfig.builder()
    .enabled(true)
    .build();
```

Resulting JSON:

```
{
  "_supertypes": ["Entity", "Auditable"]
}
```

STRUCTURED format:

```
SuperTypeSerializationConfig config = SuperTypeSerializationConfig.builder()
    .enabled(true)
    .structured()
    .build();
```

Resulting JSON:

```
{
  "_supertypes": [
    { "schema": "http://example.org/base/1.0", "name": "Entity" },
    { "schema": "http://example.org/audit/1.0", "name": "Auditable" }
  ]
}
```

With custom keys:

```
SuperTypeSerializationConfig config = SuperTypeSerializationConfig.builder()
    .enabled(true)
    .structured()
    .schemaKey("ns")
    .nameKey("type")
    .build();
```

Resulting JSON:

```
{
  "_supertypes": [
    { "ns": "http://example.org/base/1.0", "type": "Entity" },
    { "ns": "http://example.org/audit/1.0", "type": "Auditable" }
  ]
}
```

Smart compression (same-schema supertypes as plain names):

```
SuperTypeSerializationConfig config = SuperTypeSerializationConfig.builder()
    .enabled(true)
    .useSmartCompression(true)
    .build();
```

Resulting JSON: (Entity from same schema, Auditable from different)

```
{
  "_supertypes": ["Entity", "http://audit.org/1.0#//Auditable"]
}
```

Complete example output (for Person extending Entity and Auditable):

```
{
  "_supertypes": [
    { "schema": "http://example.org/base/1.0", "name": "Entity" },
    { "schema": "http://example.org/audit/1.0", "name": "Auditable" }
  ]
}
```

At serialization time:

1. Serializer reads `SuperTypeSerializationConfig` from options (Java or JSON)
2. Extracts supertypes from EObject's EClass
3. Applies smart compression (same schema → name only)
4. Combines config (keys/format) + supertype data → JSON output

8. Default Configuration

8.1 Default Strategy Settings

Target	Default Strategy	Notes
Type	URI	Full EClass URI: <code>http://example.org/person/1.0#//Person</code>
ID	ID_FIELD	Use EClass's designated ID attribute
SuperType	NONE	Disabled by default

Target	Default Strategy	Notes
Reference	STRUCTURED	With <code>_type</code> and <code>_ref</code> fields

8.2 SuperType Defaults (when enabled)

When supertype serialization is activated:

Setting	Default
Strategy	ALL (full hierarchy)
Format	ARRAY (list of values)
Value format	PLAIN (URI strings)

8.3 Default Keys

Purpose	Default Key
Type	<code>_type</code>
ID	<code>_id</code>
Reference	<code>_ref</code>
Schema	<code>_schema</code>
SuperTypes	<code>_supertypes</code>

8.4 Default Flags

Flag	Default
Smart Compression	OFF
NUMERIC mode	OFF
Include type in references	true

8.5 Default Output Example

With default settings, a Person object serializes as:

```
{
  "_type": "http://example.org/person/1.0//Person",
  "_id": "john-doe-123",
  "firstName": "John",
  "lastName": "Doe",
  "employer": {
    "_type": "http://example.org/company/1.0//Company",
```

```

    "_ref": "acme-corp"
  },
  "addresses": [
    {
      "_type": "http://example.org/person/1.0#//Address",
      "_ref": "addr-home"
    },
    {
      "_type": "http://example.org/person/1.0#//Address",
      "_ref": "addr-work"
    }
  ]
}

```

8.6 Default Configuration Code

```

// Built-in defaults - this is what you get with no configuration
CodecConfig defaults = CodecConfig.builder().build();

// Equivalent to:
// - Type: URI strategy, key="_type"
// - ID: from eID features, key="_id", PLAIN format
// - SuperType: disabled
// - Reference: PLAIN format
// - Expand: disabled
// - serializeNull=false, serializeDefaults=false, serializeEmpty=true

```

To customize, only specify non-defaults:

```

CodecConfig config = CodecConfig.builder()
    .type(TypeSerializationConfig.builder()
        .structured() // non-default: STRUCTURED instead of URI
        .build())
    .supertype(SuperTypeSerializationConfig.builder()
        .enabled(true) // non-default: enable supertypes
        .build())
    .reference(ReferenceSerializationConfig.builder()
        .structured() // non-default: STRUCTURED instead of PLAIN
        .build())
    .expand(true) // non-default: expand resolved refs
    .expandDepth(2) // non-default: 2 levels deep
    .build();

```

9. Feature Serialization

Features (EStructuralFeatures) have additional serialization options beyond the metadata targets.

9.1 Null, Default, and Empty Value Handling

Codec-Wide Defaults

Global settings that apply to all features unless overridden:

Option	Description	Default
<code>serializeDefaults</code>	Include fields with default values	<code>false</code>
<code>serializeNull</code>	Include fields with explicit null values	<code>false</code>
<code>serializeEmpty</code>	Include empty collections	<code>true</code>

Java Builder (Codec-Wide):

```
CodecConfig config = CodecConfig.builder()
    .serializeDefaults(false) // Default: omit default values
    .serializeNull(false)    // Default: omit null values
    .serializeEmpty(true)    // Default: include empty collections
    .build();
```

Default Behavior (serializeDefaults=false, serializeNull=false):

```
{
  "_type": "http://example.org/person/1.0#//Person",
  "_id": "john-doe",
  "firstName": "John",
  "lastName": "Doe"
  // "middleName" omitted (null)
  // "age" omitted (default value 0)
}
```

With serializeNull=true:

```
{
  "_type": "http://example.org/person/1.0#//Person",
  "_id": "john-doe",
  "firstName": "John",
  "middleName": null,
  "lastName": "Doe"
}
```

Feature-Specific Overrides

Override codec-wide settings for individual features:

Option	Default	Description
<code>serialize</code>	<code>true</code>	Serialize this feature (false = skip/transient)

Option	Default	Description
<code>serializeNull</code>	<code>false</code>	Include null values in output
<code>serializeDefaults</code>	<code>false</code>	Include default values in output
<code>serializeEmpty</code>	<code>true</code>	Include empty collections in output

Semantics: `true` = include in output, `false` = omit from output. Feature-level overrides codec-wide.

EAnnotation (on EStructuralFeature):

```
<!-- Mark feature as transient (never serialize) -->
<eStructuralFeatures xsi:type="ecore:EAttribute" name="internalCache">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="serialize" value="false"/>
  </eAnnotations>
</eStructuralFeatures>

<!-- Force include null values for this feature (override codec default) -->
<eStructuralFeatures xsi:type="ecore:EAttribute" name="middleName">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="serializeNull" value="true"/>
  </eAnnotations>
</eStructuralFeatures>

<!-- Skip default values for this feature -->
<eStructuralFeatures xsi:type="ecore:EAttribute" name="counter">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="serializeDefaults" value="false"/>
  </eAnnotations>
</eStructuralFeatures>
```

Annotation Details:

Key	Values	Default	Description
<code>serialize</code>	true, false	true	Serialize this feature (false = transient)
<code>serializeNull</code>	true, false	false	Include null values
<code>serializeDefaults</code>	true, false	false	Include default values
<code>serializeEmpty</code>	true, false	true	Include empty collections

Java Builder (Runtime Override):

```
// Always skip a feature (transient)
FeatureSerializationConfig transientConfig = FeatureSerializationConfig.builder()
    .feature("internalCache")
    .serialize(false) // Never serialize
    .build();
```



```
// Force serialize null for specific feature (override codec default)
FeatureSerializationConfig nullConfig = FeatureSerializationConfig.builder()
    .feature("middleName")
    .serializeNull(true) // Include even if null
    .build();

// Skip default values for specific feature
FeatureSerializationConfig defaultConfig = FeatureSerializationConfig.builder()
    .feature("counter")
    .serializeDefaults(false) // Skip if value equals default
    .build();
```

Resolution Order:

1. Feature-specific setting (if defined)
2. Codec-wide setting
3. Built-in default

9.2 Feature Key Customization

Override the JSON property name for a feature. Names can be customized at multiple levels.

Resolution Order (highest to lowest priority):

1. **Config Builder** - Runtime configuration override
2. **Model Annotation** - `@CODEC_KEY` annotation on `EStructuralFeature`
3. **Feature Name** - Default EMF feature name

Model Annotation

Customize feature names directly in the EMF model using codec annotations:

```
// In .ecore model or generated interface
@CODEC_KEY("first_name")
String getFirstName();
```

Or in the .ecore file:

```
<eStructuralFeatures xsi:type="ecore:EAttribute" name="firstName">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="key" value="first_name"/>
  </eAnnotations>
</eStructuralFeatures>
```

Resulting Output (from annotation):

```
{
  "first_name": "John",
  "lastName": "Doe"
}
```

Config Builder Override

The config builder can override annotation-defined names at runtime:

```
// Override the annotation-defined key
FeatureSerializationConfig config = FeatureSerializationConfig.builder()
    .feature("firstName")
    .key("givenName") // Overrides @CODEC_KEY("first_name")
    .build();
```

Resulting Output (config overrides annotation):

```
{
  "givenName": "John",
  "lastName": "Doe"
}
```

This allows models to define sensible defaults via annotations while still permitting runtime customization for specific use cases (e.g., different API versions, external system integration).

9.3 Extended Metadata Names

Use names from XSD extended metadata annotations instead of EMF feature names:

Java Builder:

```
CodecConfig config = CodecConfig.builder()
    .useNamesFromExtendedMetadata(true)
    .build();
```

This is useful when EMF models are generated from XSD and you want to preserve the original XML element/attribute names.

9.4 Enum Serialization

Control how enum values are serialized:

Option	Description	Example
LITERAL	Use enum literal name	"status": "ACTIVE"
VALUE	Use enum ordinal value	"status": 1
NAME	Use enum name	"status": "Active"

EAnnotation (on EEnum or EAttribute)

On the EEnum itself (applies to all usages):

```
<eClassifiers xsi:type="ecore:EEnum" name="Status">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="enumSerialization" value="LITERAL"/>
  </eAnnotations>
  <eLiterals name="ACTIVE" value="0"/>
  <eLiterals name="INACTIVE" value="1"/>
</eClassifiers>
```

Or on a specific EAttribute (overrides EEnum setting):

```
<eStructuralFeatures xsi:type="ecore:EAttribute" name="status" eType="#//Status">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="enumSerialization" value="VALUE"/>
  </eAnnotations>
</eStructuralFeatures>
```

Java Builder (Codec-Wide or Feature-Specific)

```
// Codec-wide default
CodecConfig config = CodecConfig.builder()
    .enumSerialization(EnumSerializationStrategy.LITERAL)
    .build();

// Feature-specific override
FeatureSerializationConfig featureConfig = FeatureSerializationConfig.builder()
    .feature("status")
    .enumSerialization(EnumSerializationStrategy.VALUE)
    .build();
```

10. Custom Value Readers/Writers

Extensibility hooks for custom serialization logic.

10.1 Overview

Custom value readers/writers allow you to:

- Transform values during serialization/deserialization
- Handle special data types (dates, binary, custom formats)
- Implement domain-specific encoding

10.2 Registration

Java Builder:

```
CodecConfig config = CodecConfig.builder()
    .valueWriter("dateWriter", new ISO8601DateWriter())
    .valueReader("dateReader", new ISO8601DateReader())
    .build();
```

10.3 Usage per Target

EAnnotation (on EStructuralFeature)

```
<eStructuralFeatures xsi:type="ecore:EAttribute" name="createdAt"
eType="ecore:EDatatype...">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.value.writer.name" value="dateWriter"/>
    <details key="codec.value.reader.name" value="dateReader"/>
  </eAnnotations>
</eStructuralFeatures>
```

The writer/reader must be registered in the codec configuration or be an auto-registered built-in.

Java Builder (Runtime Override)

For Type:

```
TypeSerializationConfig.builder()
    .typeWriterName("customTypeWriter")
    .typeReaderName("customTypeReader")
    .build();
```

For ID:

```
IdSerializationConfig.builder()
    .idWriterName("uuidWriter")
    .idReaderName("uuidReader")
    .build();
```

For Features:

```
FeatureSerializationConfig.builder()
    .feature("createdAt")
    .valueWriterName("dateWriter")
    .valueReaderName("dateReader")
    .build();
```

10.4 Example: Custom Date Formatting

Custom Writer:

```
public class ISO8601DateWriter implements CodecValueWriter<Date> {  
    @Override  
    public void write(Date value, JsonGenerator gen) throws IOException {  
        gen.writeString(ISO8601_FORMAT.format(value));  
    }  
}
```

Resulting Output:

```
{  
  "createdAt": "2025-12-06T10:30:00Z"  
}
```

11. Polymorphism and Inheritance

11.1 Reference Type vs Instance Type

When serializing references, the serializer decides which type information to write based on the actual instance type and smart compression setting.

Core Rule: The serializer always uses the **concrete instance type** (from `object.eClass()`), never the declared reference type.

Smart Compression Behavior:

Smart Compression	Instance Type == Reference Type	Action
ON	Yes	Omit <code>_type</code> (can be inferred)
ON	No	Write instance type
OFF	Yes	Write instance type
OFF	No	Write instance type

Examples:

```
// Reference typed as Person, contains FancyPerson instance  
EReference employeeRef; // type = Person  
EObject instance;        // eClass = FancyPerson
```

Smart Compression ON, instance type differs from reference type:

```
{
  "employee": {
    "_type": "http://example.org/person/1.0#//FancyPerson",
    "_ref": "john-doe"
  }
}
```

Smart Compression ON, instance type equals reference type:

```
{
  "employee": {
    "_ref": "john-doe"
  }
}
```

(Type omitted because it can be inferred from the reference declaration)

Smart Compression OFF (always writes instance type):

```
{
  "employee": {
    "_type": "http://example.org/person/1.0#//FancyPerson",
    "_ref": "john-doe"
  }
}
```

Option: Always Write Reference Type (Serialization-Only)

An optional setting to write the **declared reference type** instead of the instance type:

```
ReferenceSerializationConfig.builder()
    .writeReferenceType(true) // Write declared type, not instance type
    .build();
```

Result (reference typed as Person, instance is FancyPerson):

```
{
  "employee": {
    "_type": "http://example.org/person/1.0#//Person",
    "_ref": "john-doe"
  }
}
```

⚠ Warning: This is a **serialization-only option**. Using `writeReferenceType=true` may cause deserialization failures if:

- The reference type is an interface (cannot instantiate)
- The reference type is abstract (cannot instantiate)
- Instance-specific features from subclass are lost

Use only when the consuming system specifically requires the declared type.

11.2 Annotation Inheritance Levels

Controls how codec annotations are inherited across the EClass hierarchy.

Level	Description	Inherits From
DIRECT	Only direct parent	Immediate superclass/interface
ALL	Full hierarchy	All ancestors up to EObject
NONE	No inheritance	Only concrete class annotations

Default: DIRECT

Rationale for DIRECT as default:

- Avoids accidentally inheriting annotations from external library base classes
- Predictable behavior - only looks one level up
- Matches OSGi DS component annotation inheritance semantics

EAnnotation (on EClass)

```
<!-- Force inheritance from parent even if parent is from different EPackage -->
<eClassifiers xsi:type="ecore:EClass" name="Employee" eSuperTypes="#//Person">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.inherit" value="ALL"/>
  </eAnnotations>
</eClassifiers>
```

Annotation Details:

Key	Values	Default	Description
codec.inherit	DIRECT, ALL, NONE	DIRECT	Annotation inheritance level

Java Builder (Runtime Override)

```
CodecConfig config = CodecConfig.builder()
    .annotationInheritance(AnnotationInheritance.DIRECT) // Default
    .build();

// Or per-class override
ClassCodecConfig config = ClassCodecConfig.builder()
    .eClass(EmployeeClass)
    .annotationInheritance(AnnotationInheritance.ALL)
    .build();
```

Example Hierarchy:

```
EObject (EMF base)
└─ Entity (library class, has @CODEC_ID)
    └─ Person (your model, has @CODEC_TYPE)
        └─ Employee (your model, no annotations)
```

Inheritance Level	Employee sees annotations from
NONE	Employee only (none)
DIRECT	Employee + Person (@CODEC_TYPE)
ALL	Employee + Person + Entity (@CODEC_TYPE , @CODEC_ID)

11.3 Inheritance Resolution Order

When the same annotation exists at multiple levels, the most specific (closest to concrete class) wins:

1. Concrete class annotations (highest priority)
2. Direct parent annotations
3. Grandparent annotations (only if ALL)
4. Global codec defaults (lowest priority)

12. Global Configuration Options

12.1 Smart Compression

Smart Compression reduces redundancy by omitting schema information when it can be derived from context.

Behavior:

- Same-package references: use simple name instead of full URI
- Same-schema supertypes: use simple name instead of full URI
- Cross-package references: always include full URI

Configuration Levels:

```
// Global - applies to ALL serialization targets
CodecConfig.builder()
    .smartCompression(true) // Global default
    .build();

// Per-target override
CodecConfig.builder()
    .smartCompression(false) // Global default OFF
    .type(TypeSerializationConfig.builder()
        .smartCompression(true) // But ON for type info
        .build())
    .reference(ReferenceSerializationConfig.builder()
        .smartCompression(true) // And ON for references
```



```
.build())
.build();
```

Example with Smart Compression ON:

```
{
  "_type": {
    "schema": "http://example.org/company/1.0",
    "name": "Company"
  },
  "employees": [
    { "_type": "Employee", "_ref": "john" },
    { "_type": "Employee", "_ref": "jane" },
    { "_type": "http://external.org/hr/1.0#//Contractor", "_ref": "bob" }
  ],
  "headquarters": { "_type": "Address", "_ref": "addr1" }
}
```

12.2 NUMERIC/INDEXED Strategy

Uses EClass classifier IDs and EStructuralFeature IDs instead of names for maximum compactness.

Configuration:

```
// Global - applies to ALL serialization targets
CodecConfig.builder()
    .useNumericIds(true) // Global default
    .build();

// Per-target
TypeSerializationConfig.builder()
    .strategy(NUMERIC)
    .schemaKey("s") // Short keys for compactness
    .classKey("c")
    .build();

ReferenceSerializationConfig.builder()
    .strategy(NUMERIC)
    .classKey("c")
    .refKey("r")
    .build();
```

Example with NUMERIC strategy:

```
{
  "_type": { "s": "http://example.org/person/1.0", "c": 3 },
  "5": "John",
  "6": "Doe",
  "12": { "c": 1, "r": "acme" }
}
```

With smart compression (same package context):

```
{
  "_type": { "s": "http://example.org/person/1.0", "c": 3 },
  "5": "John",
  "6": "Doe",
  "12": { "c": 1, "r": "acme" },
  "15": { "s": "http://external.org/audit/1.0", "c": 2, "r": "audit-123" }
}
```

12.3 NUMERIC Strategy - Compatibility Warning

 IMPORTANT: Model Evolution Risk

The NUMERIC strategy uses EMF's internal classifier and feature IDs. These IDs are **assigned based on declaration order** in the Ecore model and **can change** when:

- A new EClass is added before existing classes
- A new EStructuralFeature is added before existing features
- Classes or features are reordered in the model
- Classes or features are removed (IDs of subsequent elements shift)

Impact:

- Serialized data using NUMERIC strategy may become **unreadable** after model changes
- The same numeric ID may refer to a **different** class/feature after model evolution
- Unlike names, numeric IDs have **no semantic stability**

Recommendations:

1. Only use NUMERIC for **transient data** (caches, message queues)
2. Do NOT use NUMERIC for **persistent storage** unless model is frozen
3. Consider using **@EClassifier(id=N)** annotations to stabilize IDs
4. Always version your serialized data format
5. For long-term storage, prefer NAME or URI strategies

Comparison:

Aspect	NAME/URI	NUMERIC
Stability	High (semantic)	Low (positional)
Compactness	Low	High
Human readable	Yes	No
Model refactoring safe	Mostly*	No
Recommended for persistence	Yes	No

*Names can also change via refactoring, but this is explicit and intentional.

13. Configuration Builder Design

13.1 Core Principle

Builders configure **FORMAT** and **KEYS**, not actual data values.

The configuration builders define:

- Which **keys** to use (`_type`, `@type`, `_id`, etc.)
- Which **format/strategy** to use (PLAIN, STRUCTURED, SCHEMA_AND_TYPE)
- Which **options** apply (include supertypes, use smart compression, etc.)

The **actual values** (schema URI, class name, ID value, etc.) come from the **EObject** at serialization time.

13.2 Configuration Object Pattern

Each serialization target has a configuration object. Only non-defaults need to be specified:

```
// Type: switch to STRUCTURED (default is URI)
TypeSerializationConfig typeConfig = TypeSerializationConfig.builder()
    .structured()
    .includeSupertypes(true)
    .useSmartCompression(true)
    .build();

// ID: default is PLAIN, only change if needed
IdSerializationConfig idConfig = IdSerializationConfig.builder()
    .structured() // switch to STRUCTURED
    .build();

// Reference: switch to STRUCTURED (default is PLAIN)
ReferenceSerializationConfig refConfig = ReferenceSerializationConfig.builder()
    .structured()
    .build();
```

13.3 Setting Configuration in Options

Configuration objects are passed via EMF's standard save/load options:

```
Map<String, Object> saveOptions = new HashMap<>();
saveOptions.put(CodecOptions.TYPE_CONFIG, typeConfig);
saveOptions.put(CodecOptions.ID_CONFIG, idConfig);
saveOptions.put(CodecOptions.REFERENCE_CONFIG, refConfig);

resource.save(outputStream, saveOptions);
```

13.4 Serializer Flow

At serialization time:

1. Serializer receives EObject
2. Serializer reads configuration from options (keys, format)
3. Serializer extracts values from EObject (schema, name, id, etc.)
4. Serializer combines: config.keys + eObject.values → JSON output

Example for type serialization:

```
// Serializer pseudocode
void serializeType(EObject obj, JsonGenerator gen, TypeSerializationConfig config) {
    EClass eClass = obj.eClass();

    // Extract VALUES from EObject
    String schemaUri = eClass.getEPackage().getNsURI();
    String className = eClass.getName();
    List<String> supertypes = getSupertypes(eClass, config.useSmartCompression());

    // Use KEYS from config
    if (config.strategy() == STRUCTURED) {
        gen.writeObjectFieldStart(config.typeKey()); // "_type"
        gen.writeStringField(config.schemaKey(), schemaUri); // "schema"
        gen.writeStringField(config.nameKey(), className); // "name"
        if (config.includeSupertypes()) {
            gen.writeArrayFieldStart(config.supertypesKey()); // "supertypes"
            supertypes.forEach(gen::writeString);
            gen.writeEndArray();
        }
        gen.writeEndObject();
    } else {
        // PLAIN: just write URI
        gen.writeStringField(config.typeKey(), eClass.getURI());
    }
}
```

13.5 Default Configurations

Codec provides factory methods for common presets:

```
// Default (URI strategy) - what you get with no config
TypeSerializationConfig.builder().build();

// JSON-LD preset
TypeSerializationConfig.builder()
    .schemaAndType()
    .typeKey("@type")
    .schemaKey("@context")
    .build();
```

```
// Fully structured preset
TypeSerializationConfig.builder()
    .structured()
    .includeSupertypes(true)
    .build();
```

14. Complete Configuration Scenarios

This section shows 4 complete scenarios with varying complexity. Each scenario includes:

- The `CodecConfig` builder configuration (only non-defaults shown)
- The resulting serialized JSON output for a sample Person object

Sample EObject for all scenarios:

- Person (from `http://example.org/person/1.0`) extends Entity (from `http://example.org/base/1.0`)
- ID: "john-doe-123"
- firstName: "John", lastName: "Doe"
- employer: reference to Company "acme-corp"
- addresses: list of Address references ["addr-home", "addr-work"]

14.1 Scenario 1: Minimal (Default Configuration)

Uses all default settings - no configuration needed.

Java Builder:

```
CodecConfig config = CodecConfig.builder().build();
```

Resulting Output: (Type=URI, ID=PLAIN, Reference=PLAIN, SuperType=disabled)

```
{
  "_type": "http://example.org/person/1.0//Person",
  "_id": "john-doe-123",
  "firstName": "John",
  "lastName": "Doe",
  "employer": "acme-corp",
  "addresses": ["addr-home", "addr-work"]
}
```

14.2 Scenario 2: Standard API with SuperTypes

Standard JSON API style with supertype information and STRUCTURED references.

Java Builder:

```

CodecConfig config = CodecConfig.builder()
    .reference(ReferenceSerializationConfig.builder()
        .structured() // STRUCTURED references
        .build())
    .supertype(SuperTypeSerializationConfig.builder()
        .enabled(true) // Enable supertypes
        .build())
    .build();

```

Resulting Output:

```

{
  "_type": "http://example.org/person/1.0#//Person",
  "_id": "john-doe-123",
  "_supertypes": ["http://example.org/base/1.0#//Entity"],
  "firstName": "John",
  "lastName": "Doe",
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_ref": "acme-corp"
  },
  "addresses": [
    { "_type": "http://example.org/person/1.0#//Address", "_ref": "addr-home" },
    { "_type": "http://example.org/person/1.0#//Address", "_ref": "addr-work" }
  ]
}

```

14.3 Scenario 3: JSON-LD / Semantic Web Style

Optimized for JSON-LD compatibility with @ prefixed keys and schema/type separation.

Java Builder:

```

CodecConfig config = CodecConfig.builder()
    .type(TypeSerializationConfig.builder()
        .schemaAndType() // separate schema + type keys
        .schemaKey("@context")
        .typeKey("@type")
        .build())
    .id(IdSerializationConfig.builder()
        .idKey("@id")
        .build())
    .reference(ReferenceSerializationConfig.builder()
        .structured()
        .typeKey("@type")
        .refKey("@ref")
        .build())
    .supertype(SuperTypeSerializationConfig.builder()
        .enabled(true)
        .supertypesKey("@supertypes")
        .useSmartCompression(true)
    )

```

```
.build())  
.build();
```

Resulting Output:

```
{  
  "@context": "http://example.org/person/1.0",  
  "@type": "Person",  
  "@id": "john-doe-123",  
  "@supertypes": ["http://example.org/base/1.0#//Entity"],  
  "firstName": "John",  
  "lastName": "Doe",  
  "employer": {  
    "@context": "http://example.org/company/1.0",  
    "@type": "Company",  
    "@ref": "acme-corp"  
  },  
  "addresses": [  
    { "@type": "Address", "@ref": "addr-home" },  
    { "@type": "Address", "@ref": "addr-work" }  
  ]  
}
```

14.4 Scenario 4: Fully Structured with Smart Compression

Maximum detail with structured objects and smart compression for same-schema references.

Java Builder:

```
CodecConfig config = CodecConfig.builder()  
    .type(TypeSerializationConfig.builder()  
        .structured()  
        .build())  
    .id(IdSerializationConfig.builder()  
        .structured()  
        .build())  
    .reference(ReferenceSerializationConfig.builder()  
        .structured()  
        .build())  
    .supertype(SuperTypeSerializationConfig.builder()  
        .enabled(true)  
        .structured()  
        .build())  
    .useSmartCompression(true)  
    .build();
```

Resulting Output:

```
{  
  "_type": {  
    "schema": "http://example.org/person/1.0",
```

```

    "name": "Person"
  },
  "_id": {
    "type": "http://example.org/person/1.0//Person",
    "value": "john-doe-123"
  },
  "_supertypes": [
    { "schema": "http://example.org/base/1.0", "name": "Entity" }
  ],
  "firstName": "John",
  "lastName": "Doe",
  "employer": {
    "_type": {
      "schema": "http://example.org/company/1.0",
      "name": "Company"
    },
    "_ref": "acme-corp"
  },
  "addresses": [
    { "_type": "Address", "_ref": "addr-home" },
    { "_type": "Address", "_ref": "addr-work" }
  ]
}

```

Note: Smart compression applies to `addresses` - since Address is from the same schema as Person, only the name is used instead of the full structured type.

15. Deserialization Requirements

15.1 Format Detection

Deserializers MUST detect the format used and handle accordingly:

Input	Detection
<code>"_type": "string"</code>	PLAIN (detect NAME/CLASS/URI/MAPPED by content)
<code>"_type": { ... }</code>	STRUCTURED
<code>"_schema": ..., "_type": ...</code>	SCHEMA_AND_TYPE

15.2 Backward Compatibility

Deserializers MUST support reading:

- V1 format (always PLAIN)
- V2 PLAIN format
- V2 STRUCTURED format
- V2 SCHEMA_AND_TYPE format

The deserializer should auto-detect the format regardless of codec configuration.

15.3 Type Resolution

For deserialization without ROOT_OBJECT:

1. Read type information (any format)
2. Extract schema/namespace URI
3. Lookup EPackage in ModelInfoService
4. Resolve EClass by name
5. Create and populate EObject

16. Configuration Hierarchy

16.1 Levels (Dynamic before Static)

Priority	Level	Scope
1 (highest)	Load/Save options	Per-operation (<code>resource.save(options)</code>)
2	ResourceFactory defaults	Per-factory (<code>defaultSaveOptions</code> / <code>defaultLoadOptions</code>)
3	Codec module config	Per-codec (Jackson module configuration)
4	Configuration properties	External (system props, config files)
5	EAnnotations	Per-model (declared in .ecore)
6 (lowest)	Built-in defaults	Global (hardcoded)

16.2 Override Rules

- Dynamic/runtime settings override static/declared settings
- Every EAnnotation detail key has a corresponding runtime option key
- Every Jackson module feature has a corresponding runtime option key
- This allows complete override capability at load/save time

17. Test Scenarios

17.1 Serialization Tests

ID	Scenario	Input	Expected Output
S1	Type PLAIN/URI	Person object	<code>"_type": "http://...#//Person"</code>
S2	Type PLAIN/NAME	Person object	<code>"_type": "Person"</code>

ID	Scenario	Input	Expected Output
S3	Type STRUCTURED	Person object	<code>"_type": { "schema": "...", "name": "Person" }</code>
S4	Type SCHEMA_AND_TYPE	Person object	<code>"_schema": "...", "_type": "Person"</code>
S5	ID PLAIN	Person with id	<code>"_id": "john"</code>
S6	ID STRUCTURED	Person with id	<code>"_id": { "type": "...", "value": "john" }</code>
S7	Ref PLAIN	Person with employer	<code>"employer": "acme"</code>
S8	Ref STRUCTURED	Person with employer	<code>"employer": { "_type": "...", "_ref": "acme" }</code>
S9	SuperType PLAIN	Person extends Entity	<code>"_supertypes": ["...#//Entity"]</code>
S10	SuperType STRUCTURED	Person extends Entity	<code>"_supertypes": [{ "schema": "...", "name": "Entity" }]</code>
S11	JSON-LD keys	Person with @type/@id	<code>"@type": "Person", "@id": "john"</code>
S12	Mixed strategies	Type STRUCTURED, Ref PLAIN	Combination

17.2 Deserialization Tests

ID	Scenario	Input	Expected
D1	Type PLAIN/URI	<code>"_type": "http://...#//Person"</code>	Person EObject
D2	Type STRUCTURED	<code>"_type": { "schema": "...", "name": "Person" }</code>	Person EObject
D3	Type SCHEMA_AND_TYPE	<code>"_schema": "...", "_type": "Person"</code>	Person EObject
D4	Auto-detect format	Any valid format	Correct EObject
D5	Unknown package	Type with unregistered schema	Error or lazy load
D6	Ref resolution	<code>"employer": { "_ref": "acme" }</code>	Resolved reference

17.3 Round-trip Tests

ID	Scenario	Verify
R1	PLAIN round-trip	serialize → deserialize → equals
R2	STRUCTURED round-trip	serialize → deserialize → equals
R3	Cross-format	serialize PLAIN → deserialize (auto) → equals
R4	V1 compatibility	V1 output → V2 deserialize → equals

18. Additional Serialization Features

18.1 Collection and Array Handling

Ordered vs Unordered Collections

EMF Type	JSON Representation	Notes
<code>EList</code>	JSON array	Order preserved
<code>ESet</code> (unique)	JSON array	Order preserved, uniqueness enforced on load

Multi-Dimensional Arrays

For complex data like GeoJSON coordinates, multi-dimensional arrays are supported via `EDataTypes`:

```
// EDataType for 2D coordinates
EDataType coordinateArray = EcoreFactory.eINSTANCE.createEDataType();
coordinateArray.setInstanceClass(double[][].class);
```

Resulting Output:

```
{
  "type": "Polygon",
  "coordinates": [
    [[0, 0], [10, 0], [10, 10], [0, 10], [0, 0]]
  ]
}
```

18.2 EMap Serialization

EMF Maps (`EMap<K, V>`) are serialized as JSON objects when keys are strings, or as arrays of key-value pairs for complex keys:

String Keys (default):

```
{
  "properties": {
    "name": "John",
    "age": "30"
  }
}
```

Complex Keys (EObject keys):

```
{
  "associations": [
    { "_key": { "_type": "...", "_ref": "person1" }, "_value": "manager" },
    { "_key": { "_type": "...", "_ref": "person2" }, "_value": "employee" }
  ]
}
```

Configurable keys:

- `mapKeyKey`: key field name (default: `_key`)
- `mapValueKey`: value field name (default: `_value`)

18.3 Null vs Absent vs Default

See **Section 9.1** for complete documentation of null, default, and empty value handling.

Summary:

- Codec-wide defaults: `serializeDefaults=false`, `serializeNull=false`, `serializeEmpty=true`
- Feature-specific overrides: `serialize`, `serializeNull`, `serializeDefaults`, `serializeEmpty`

18.4 Proxy and Reference Expansion

Default Behavior: Serialize as Proxy

Non-containment references are serialized as **proxies by default**. At deserialization, they remain proxies until explicitly resolved.

Rationale:

- At deserialization time, we don't know if resolution is needed
- Referenced data may not be accessible
- Lazy loading is more efficient

Default Output:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_ref": "acme-corp"
  }
}
```

Expand Option: Full Object Serialization

For specific features, expand non-containment references to full objects:

EAnnotation (on EReference):

```
<eStructuralFeatures xsi:type="ecore:EReference" name="employer" eType="#//Company">
  <eAnnotations source="http://eclipse.org/fennec/codec">
    <details key="codec.reference"/>
    <details key="expand" value="true"/>
  </eAnnotations>
</eStructuralFeatures>
```

Java Builder (Runtime Override):

```
FeatureSerializationConfig config = FeatureSerializationConfig.builder()
    .feature("employer")
    .expand(true) // Serialize full object, not just reference
    .build();
```

With expand=true:

```
{
  "employer": {
    "_type": "http://example.org/company/1.0#//Company",
    "_id": "acme-corp",
    "name": "ACME Corporation",
    "address": "123 Main St"
  }
}
```

Note: Expanded objects create a copy in the JSON. Changes to the expanded object don't affect the original. Use with caution.

18.5 Unknown Fields Handling

Options for handling fields in JSON that don't match EClass features:

Option	Behavior
IGNORE	Silently skip unknown fields
STORE	Collect in map, return via Resource options
FAIL	Throw exception

Default: IGNORE

Java Builder:

```
CodecConfig config = CodecConfig.builder()
    .unknownFieldHandling(UnknownFieldHandling.STORE)
    .build();
```

Retrieving Stored Unknown Fields:

```
Resource resource = resourceSet.getResource(uri, true);
Map<String, Object> loadOptions = resource.getDefaultLoadOptions();

// After loading, unknown fields are available in options
@SuppressWarnings("unchecked")
Map<EObject, Map<String, Object>> unknownFields =
    (Map<EObject, Map<String, Object>>) loadOptions.get(CodecOptions.UNKNOWN_FIELDS);
```

18.6 Field Ordering

Deterministic field ordering can improve deserialization performance.

Default Order (when enabled):

1. `_type`
2. `_supertypes`
3. `_id`
4. Features in EClass declaration order

Java Builder:

```
CodecConfig config = CodecConfig.builder()
    .deterministicFieldOrder(true) // Enable ordered output
    .build();
```

Performance Note: When field order matches expected order, the deserializer can use direct mapping without cache lookups, significantly improving throughput for large datasets.

18.7 Root Element / Multiple Contents

EMF Resources support multiple root elements in the contents list. This maps to JSON arrays:

Single Root (default for single content):

```
{
  "_type": "http://example.org/person/1.0#//Person",
  "_id": "john",
  "firstName": "John"
}
```

Multiple Roots (for multiple contents):

```
[
  {
    "_type": "http://example.org/person/1.0#//Person",
    "_id": "john",
    "firstName": "John"
  },
  {
    "_type": "http://example.org/person/1.0#//Person",
    "_id": "jane",
    "firstName": "Jane"
  }
]
```

Deserialization: Automatically detects array vs object at root.

18.8 Circular References (Future)

Note: This feature is designed but not actively implemented in v2.0.

For bidirectional references that create cycles, the metadata service can track this:

```
// In metadata aspect
FeatureCodecAspect aspect = ...;
aspect.isBidirectional(); // true if this creates potential cycles
```

Handling strategies (future):

- First occurrence: full object
- Subsequent occurrences: reference only
- Detection via object identity tracking during serialization

18.9 Custom EDataType Serialization

For custom EDataTypes beyond standard primitives, register Jackson serializers/deserializers:

Via Codec API:

```
CodecConfig config = CodecConfig.builder()
    .registerSerializer(BigDecimal.class, new BigDecimalSerializer())
    .registerDeserializer(BigDecimal.class, new BigDecimalDeserializer())
    .build();
```

Via Metadata API (for model-wide defaults):

```
// On EDataType annotation
@CODEC_SERIALIZER("bigDecimalSerializer")
@CODEC_DESERIALIZER("bigDecimalDeserializer")
EDataType myBigDecimal;
```

18.10 Jackson 3 Integration

The following features delegate to Jackson 3 configuration and are not duplicated in codec config:

- **Binary data encoding** (Base64)
- **Date/Time formatting** (ISO8601, timestamps)
- **Pretty printing** (indentation, formatting)
- **Number precision** (BigDecimal scale)

Configuration via ObjectMapperConfigurator:

```
ObjectMapperConfigurator config = ObjectMapperConfigurator.builder()
    .enable(SerializationFeature.INDENT_OUTPUT)
    .dateFormat(new ISO8601DateFormat())
    .build();
```

Only features NOT supported by Jackson 3 need codec-specific handling.

19. Open Questions

1. **Supertype in STRUCTURED type** ✓ ANSWERED
→ Separate config via `includeSupertypes(true)` on `TypeSerializationConfig`. Not automatic.
2. **Reference type inference** ✓ ANSWERED
→ Configurable via `useSmartCompression(true)`. If all refs in collection have same type, omit type info.
3. **Null handling in STRUCTURED format** ✓ ANSWERED
→ Follow `serializeNull` setting. If included, output `"employer": null` (not a structured null object).
4. **Proxy references** ✓ ANSWERED → See Section 5.4 (Proxy and Expand Handling)
→ Covers proxy serialization, expand options, type config per context, bi-directional handling.
5. **Cross-document references** ✓ ANSWERED
→ Use URIs as-is. No special treatment (full vs relative) for now.

20. Spec Contradictions and Issues to Resolve

20.1 PLAIN Strategy Terminology ✓ RESOLVED

Issue: The term "PLAIN" was used inconsistently - sometimes as a strategy, sometimes as a category.

Resolution: Added Section 2.1 "Plain vs Structured Output Format" as the central definition:

- **Plain:** Property value is a simple type (string, number, boolean, array)
- **Structured:** Property value is a nested object/map

Section 2.2 now classifies all strategies by output format. Section 3.1 now groups type strategies as "Plain Strategies" and "Structured Strategies" with clear headings.

20.2 ID Strategy vs Format Confusion ✓ RESOLVED

Issue: Section 4 conflated ID source (ID_FIELD vs COMBINED) with ID format (PLAIN vs STRUCTURED).

Resolution: Simplified to two orthogonal dimensions:

- **ID Key Mode:** ID_ONLY | BOTH | FEATURE_ONLY - how to represent in JSON
- **ID Format:** PLAIN | STRUCTURED - per Section 2.1

The "COMBINED" strategy is now implicit - when `idFeatures` has multiple entries, values are automatically combined with separator. Section 4 completely rewritten with examples from `org.eclipse.fennec.codec.v2.example/model/test.ecore`.

20.3 ignoreNull vs serializeNull Semantics ✓ RESOLVED

Issue: Confusing opposite naming:

- Codec-wide: `serializeNull=false` → don't serialize nulls
- Feature-level: `ignoreNull=false` → don't ignore = DO serialize nulls

Same boolean value has opposite meanings!

Resolution: Use `serializeX` consistently everywhere:

- `serialize` (default: true) - false = transient/skip feature
- `serializeNull` (default: false) - true = include null values
- `serializeDefaults` (default: false) - true = include default values
- `serializeEmpty` (default: true) - true = include empty collections

Semantics: `true` = include in output, `false` = omit from output. Feature-level overrides codec-wide. See Section 9.1 for examples.

20.4 Feature Key Annotation Name ✓ RESOLVED

Issue:

- Section 1.1 lists `codec.key` as the annotation
- Section 9.2 shows `<details key="key" value="first_name"/>`

Resolution: Section 1.1 table updated. Within the EAnnotation (source: <http://eclipse.org/fennec/codec>), the detail key is simply `key`, not `codec.key`. The `codec.` prefix is only used for marker annotations like `codec.type`, `codec.id`, etc.

20.5 SuperType strategy vs format ✓ RESOLVED

Issue: Section 7.3 Java Builder shows both `.strategy(...)` and `.format(...)` but their relationship is unclear.

Resolution: Section 7 updated with clear terminology:

- `selection`: ALL | ALL_EMF | SINGLE | NONE - which supertypes to include
- `format`: PLAIN | STRUCTURED - per Section 2.1, how each entry is represented

Added `ALL_EMF` to include EMF base classes (EObject, etc.) when needed. Default `ALL` excludes EMF internal classes. Builder uses `.structured()` method to switch to STRUCTURED format (PLAIN is default).

20.6 Configuration Hierarchy Inconsistency ✓ RESOLVED

Issue:

- Section 1.1: "Runtime Config Builder > Model EAnnotations > Codec Defaults" (3 levels)
- Section 16.1: "Global defaults > Codec instance > Per-EClass > Runtime options" (4 levels)

Resolution: Consolidated to a 6-level hierarchy (dynamic before static):

1. Load/Save options (per-operation)
2. ResourceFactory defaults (per-factory)
3. Codec module config (per-codec)
4. Configuration properties (external)
5. EAnnotations (per-model, static)
6. Built-in defaults (global)

Key principle: Every configuration setting (EAnnotation keys, Jackson features) must have a corresponding runtime option key for override capability.

20.7 Implementation Checklist Outdated ✓ RESOLVED

Issue: Section 21 checklist shows items already completed (TypeStrategy enum update).

Resolution: Section 21 now contains implementation strategy and phased checklist.

21. Implementation Strategy

21.1 Approach: Minimal Pipeline First

The implementation follows this approach:

1. **Minimal working pipeline** - ResourceFactory → Resource → ObjectMapper → JSON
2. **Test-driven** - Write tests as we build each layer
3. **Feature-by-feature** - Add one feature at a time, fully tested
4. **Reference v1** - Always compare with existing codec behavior

21.2 Implementation Phases

Phase 1: Infrastructure (Minimal Pipeline)

Goal: Get a minimal JSON output from an EObject with hardcoded defaults.

Steps:

1. Create `org.eclipse.fennec.codec.v2` project structure
2. Define `CodecResourceFactory` interface

3. Create `CodecResource` that wraps Jackson ObjectMapper
4. Implement basic `CodecEObjectSerializer` (type + features only)
5. **Test:** Serialize simple Person → JSON with `_type` and feature values

Checklist per step:

- ☐ Does v1 have this? How?
- ☐ What configuration does v1 support here?
- ☐ Test written before implementation?
- ☐ Test passes?

Phase 2: Deserialization Pipeline

Goal: Round-trip - serialize and deserialize back to equal EObject.

Steps:

1. Implement `CodecEObjectDeserializer`
2. Implement type resolution (lookup EClass from `_type`)
3. Implement feature deserialization
4. **Test:** Round-trip Person → JSON → Person with EMF equals

Phase 3: Configuration System

Goal: Make the pipeline configurable.

Steps:

1. Define `CodecConfig` and target-specific config builders
2. Wire config into serializers/deserializers
3. Implement strategy selection (use config to pick serialization approach)
4. **Test:** Same EObject with different configs → different JSON

Phase 4: Type Strategies

Goal: Support all type serialization strategies.

Steps (one strategy at a time):

1. URI (default) - already in Phase 1
2. NAME - simple class name
3. CLASS - instance class name
4. STRUCTURED - nested object
5. SCHEMA_AND_TYPE - separate fields
6. MAPPED - discriminator-based
7. NUMERIC - classifier IDs

For each strategy:

- ☐ Serialization test
- ☐ Deserialization test (with auto-detection)
- ☐ Round-trip test

Phase 5: ID Strategies

Goal: Support all ID serialization strategies.

Steps:

1. ID_FIELD source + PLAIN format (default)
2. COMBINED source + PLAIN format
3. STRUCTURED format
4. NUMERIC format
5. Custom ID value readers/writers

Phase 6: Reference Strategies

Goal: Support all reference serialization strategies.

Steps:

1. PLAIN (default - ID only)
2. STRUCTURED (type + ref)
3. NUMERIC
4. Smart compression (omit type when inferable)
5. Expand option (full object)

Phase 7: SuperType Serialization

Goal: Support supertype serialization (disabled by default).

Steps:

1. Enable/disable flag
2. ALL vs SINGLE strategy
3. PLAIN format (URIs)
4. STRUCTURED format
5. Smart compression for same-schema

Phase 8: Feature Options

Goal: Complete feature-level customization.

Steps:

1. Null/Default/Empty handling
2. Feature key customization
3. Transient features

4. Custom value readers/writers
5. Enum serialization strategies
6. Extended metadata names

Phase 9: Advanced Features

Goal: Complete remaining features.

Steps:

1. Cross-document containment
2. EMap serialization
3. Multi-dimensional arrays
4. Unknown fields handling
5. Field ordering
6. Multiple root elements

21.3 Feature Checklist Template

For each feature implementation, verify:

Check	Description
V1 Behavior	How does v1 handle this? Document any differences
Spec Alignment	Does implementation match spec? Note any deviations
EAnnotation	Can it be configured via annotation?
Config Builder	Can it be configured at runtime?
Default Value	What's the default? Matches spec?
Serialization Test	Written and passing?
Deserialization Test	Written and passing (with auto-detect)?
Round-trip Test	Written and passing?
Edge Cases	Null, empty, missing values handled?

21.4 V1 Reference Points

Key v1 classes to reference during implementation:

Component	V1 Location	Purpose
ResourceFactory	<code>org.eclipse.fennec.codec.json</code>	How factory creates resources
Resource	<code>CodecResource</code>	How resource wraps ObjectMapper
ObjectMapper setup	Configurators	How Jackson is configured

Component	V1 Location	Purpose
EObject serializer	<code>CodecEObjectSerializer</code>	Main serialization logic
Type serializer	<code>CodecTypeInfoSerializer</code>	Type info handling
ID serializer	<code>CodecIdInfoSerializer</code>	ID handling
Reference serializer	Various	Reference handling
Feature serializer	<code>CodecFeatureInfoSerializer</code>	Feature handling
Deserializers	<code>*Deserializer</code> classes	Deserialization logic

22. Implementation Checklist

Metadata Layer (codec.metadata) - Completed

- ☒ Add `STRUCTURED` to `TypeStrategy` enum
- ☒ Add `SCHEMA_AND_TYPE` to `TypeStrategy` enum
- ☒ Add `NUMERIC` to `TypeStrategy` enum
- ☒ Add `MAPPED` to `TypeStrategy` enum
- ☒ Add `SuperTypeSelection` enum (ALL, ALL_EMF, SINGLE, NONE)
- ☐ Add `IdKeyMode` enum (ID_ONLY, BOTH, FEATURE_ONLY)
- ☐ Add `IdFormat` enum (PLAIN, STRUCTURED)
- ☐ Add `ReferenceFormat` enum (PLAIN, STRUCTURED, NUMERIC)
- ☐ Add builder methods: `.structured()`, `.schemaAndType()`, etc.
- ☐ Update `TypeResolutionInfo` for new strategies
- ☐ Define `ClassCodecAspect` interface
- ☐ Define `FeatureCodecAspect` interface

Phase 1: Infrastructure (Minimal Pipeline)

- ☐ Create `org.eclipse.fennec.codec.v2` project with Gradle setup
- ☐ Define `CodecResourceFactory` interface
- ☐ Create `CodecResource` implementation
- ☐ Create Jackson `CodecModule` with basic serializers
- ☐ Implement `CodecEObjectSerializer` (minimal: type URI + features)
- ☐ **TEST:** Serialize Person → JSON with `_type` and features

Phase 2: Deserialization

- ☐ Implement `CodecEObjectDeserializer` (minimal)
- ☐ Implement type resolution (URI → EClass lookup)
- ☐ Implement feature deserialization
- ☐ **TEST:** Deserialize JSON → Person
- ☐ **TEST:** Round-trip Person → JSON → Person equals

Phase 3: Configuration System

- ☐ Define `CodecConfig` interface and builder
- ☐ Define `TypeSerializationConfig` builder
- ☐ Wire config into serializers/deserializers
- ☐ **TEST:** Same EObject + different config → different JSON

Phase 4: Type Strategies

Strategy	Serialization	Deserialization	Round-trip
URI	[]	[]	[]
NAME	[]	[]	[]
CLASS	[]	[]	[]
STRUCTURED	[]	[]	[]
SCHEMA_AND_TYPE	[]	[]	[]
MAPPED	[]	[]	[]
NUMERIC	[]	[]	[]

Phase 5: ID Strategies

- ☐ `IdSerializationConfig` builder
- ☐ ID_FIELD source + PLAIN format
- ☐ COMBINED source + PLAIN format
- ☐ STRUCTURED format
- ☐ NUMERIC format
- ☐ Custom ID value readers/writers
- ☐ **TESTS:** All combinations

Phase 6: Reference Strategies

- ☐ `ReferenceSerializationConfig` builder
- ☐ PLAIN format (default)
- ☐ STRUCTURED format
- ☐ NUMERIC format
- ☐ Smart compression
- ☐ Expand option
- ☐ **TESTS:** All combinations

Phase 7: SuperType Serialization

- ☐ `SuperTypeSerializationConfig` builder
- ☐ Enable/disable flag
- ☐ ALL vs SINGLE strategy
- ☐ PLAIN format
- ☐ STRUCTURED format
- ☐ Smart compression
- ☐ **TESTS:** All combinations

Phase 8: Feature Options

- ☐ `FeatureSerializationConfig` builder
- ☐ Null handling (codec-wide + feature-specific)
- ☐ Default value handling
- ☐ Empty collection handling
- ☐ Feature key customization
- ☐ Transient features (serialize=false)
- ☐ Custom value readers/writers
- ☐ Enum serialization strategies
- ☐ Extended metadata names
- ☐ **TESTS:** All options

Phase 9: Advanced Features

- ☐ Cross-document containment
- ☐ EMap serialization (string keys + complex keys)
- ☐ Multi-dimensional arrays
- ☐ Unknown fields handling (IGNORE, STORE, FAIL)
- ☐ Field ordering
- ☐ Multiple root elements

☐ **TESTS:** All features

Compatibility & Performance

- ☐ V1 format compatibility tests
- ☐ Auto-detect format on deserialization
- ☐ Model evolution tests (NUMERIC stability)
- ☐ Performance benchmarks (size comparison)
- ☐ Performance benchmarks (throughput)